

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа №1 по курсу "Методы вычислений"

Тема Венгерский метод решения задачи о назначениях

Студент Нисуев Н. Ф.

Группа ИУ7-11М

Вариант 8

Преподаватель Власов П. А.

Москва — 2026 г.

СОДЕРЖАНИЕ

1 Содержание работы **3**

2 Теоретическая часть **4**

2.1 Содержательная и математическая постановки задачи о назаначениях 4

2.2 Краткое описание венгерского метода 5

3 Практическая часть **7**

3.1 Исходные данные конкретного варианта 7

3.2 Текст программы 7

3.3 Результаты работы программы 16

1 Содержание работы

Цель работы: изучение венгерского метода решения задачи о назначениях.

1. реализовать венгерский метод решения задачи о назначениях в виде программы на ЭВМ;

2. провести решение задачи с матрицей стоимостей, заданной в индивидуальном варианте, рассмотрев два случая:

   а) задача о назначениях является задачей минимизации,

   б) задача о назначениях является задачей максимизации.

В программе необходимо предусмотреть два режима работы: «итоговый», когда программа печатает только матрицу назначений, и «отладочный», когда на экран поитерационно выводится вспомогательная информация о текущей эквивалентной матрице стоимостей, текущей системе независимых нулей, выделенных столбцах и строках, нулях со штрихами, L-цепочке и т.д.

2 Теоретическая часть

2.1 Содержательная и математическая постановки задачи о назаначениях

Содержательная постановка: В распоряжении работодателя имеется n работ и n исполнителей. Стоимость выполнения i-ой работы j-ым исполнителем составляет $c_{ij} \geqslant 0$ единиц. Требуется распределить все работы между исполнителями так, чтобы:

1) каждый исполнитель выполнил ровно 1 работу;

2) общая стоимость выполнения всех работ была min.

Величины c_{ij} принято записывать в матрицу $C = (c_{ij}),\ i, j = \overline{1,n}$, которая называется матрицей стоимостей.

Введем управляемые переменные:

$$x_{ij} = \begin{cases} 1, & \text{если } i\text{-ую работу выполняет } j\text{-ый работник,} \\ 0, & \text{иначе,} \end{cases}$$

где $i, j = \overline{1,n}$.

Управляемые переменные x_{ij} записывают в матрицу $X = (x_{ij}),\ i, j = \overline{1,n}$, которая называется матрицей назначений.

Тогда:

1) Общая стоимость выполнения всех работ:

$$f(x) = \sum_{i=0}^{n} \sum_{j=0}^{n} c_{ij} x_{ij}.$$

2) Условие того что i-ую работу выполняет один сотрудник:

$$\sum_{j=0}^{n} x_{ij} = 1,\ i = \overline{1,n}.$$

3) Условие того что j-ый работник выполняет ровно одну работу:

$$\sum_{i=0}^{n} x_{ij} = 1,\ j = \overline{1,n}.$$

Таким образом приходим к **математической постановке** задачи о назначениях:

$$\begin{cases} f = \sum\limits_{i=1}^{n}\sum\limits_{j=1}^{n} c_{ij}x_{ij} \to \min, \\ \sum\limits_{j=1}^{n} x_{ij} = 1, \quad i = \overline{1,n}, \\ \sum\limits_{i=1}^{n} x_{ij} = 1, \quad j = \overline{1,n}, \\ x_{ij} \in \{0,1\},\ i,j = \overline{1,n}. \end{cases}$$

2.2 Краткое описание венгерского метода

Схема венгерского метода решения задачи о назаначениях представлена на рисунках 2.1 — 2.2.

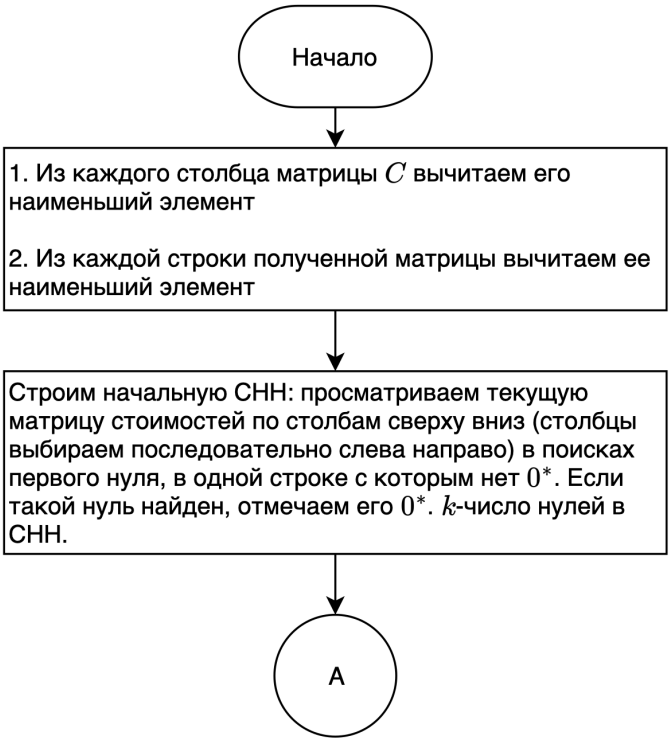


Рисунок 2.1 – Схема подготовительного этапа венгерского метода

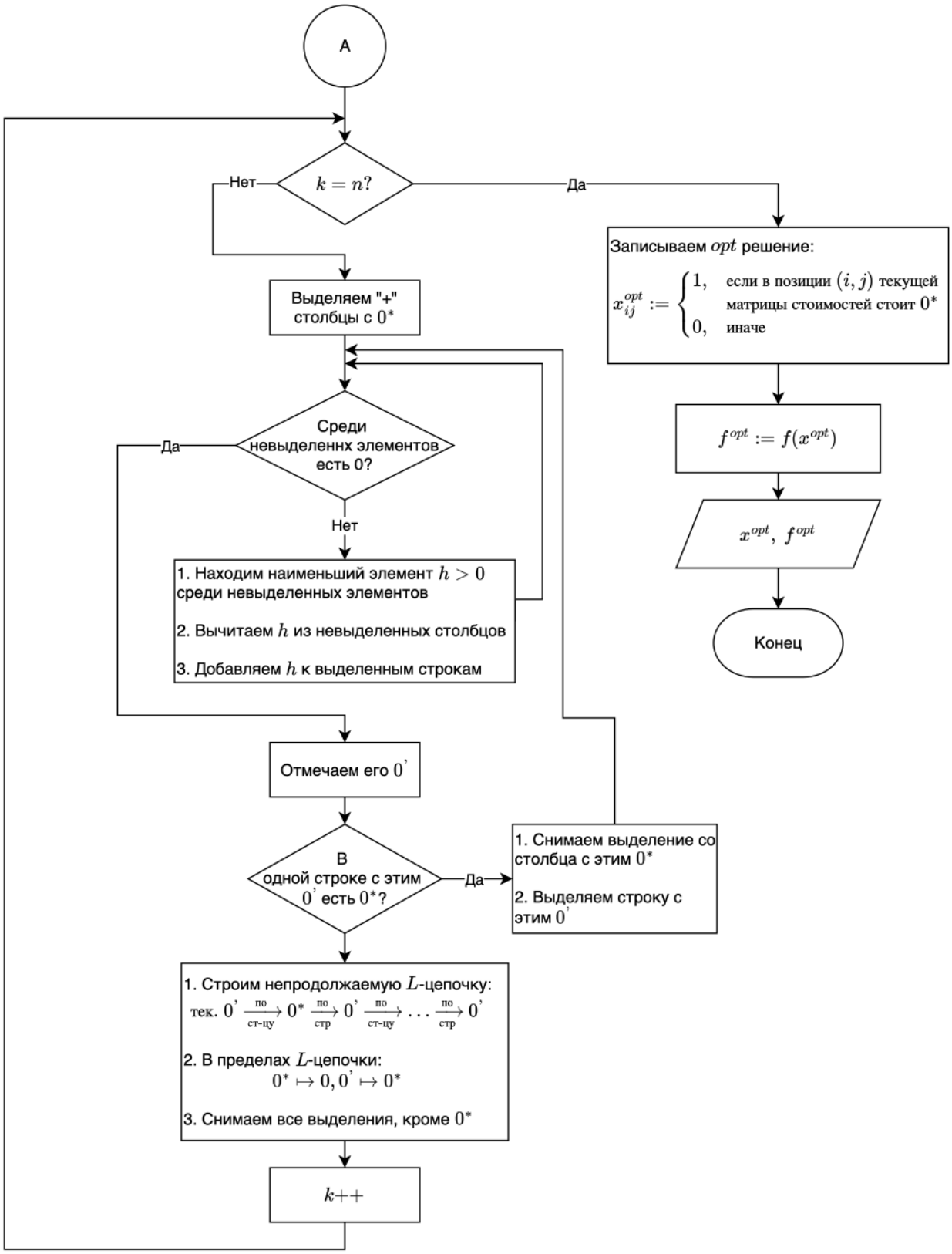


Рисунок 2.2 – Схема основного этапа венгерского метода

3 Практическая часть

3.1 Исходные данные конкретного варианта

Матрица стоимостей варианта **№8**:

$$C = \begin{bmatrix} 7 & 4 & 3 & 8 & 2 \\ 4 & 5 & 1 & 6 & 3 \\ 8 & 4 & 5 & 7 & 2 \\ 1 & 2 & 4 & 7 & 2 \\ 3 & 9 & 9 & 2 & 5 \end{bmatrix}.$$

3.2 Текст программы

Текст программы представлен в листинге 3.1.

Листинг 3.1 – Текст программы

```
is_find_max = false;
is_debug = true;

mtx = [
    7 4 3 8 2;
    4 5 1 6 3;
    8 4 5 7 2;
    1 2 4 7 2;
    3 9 9 2 5
];

venger_method(mtx, is_find_max, is_debug)

function venger_method(srcC, is_find_max, is_debug)
    fprintf('Матрица стоимостей:\n');
    C = srcC;
    disp(C);

    if is_find_max
        C = reduction_to_minimization(C, is_debug);
    end

```

Продолжение листинга 3.1

```
23      [C, SIZ] = prepare_step(C, is_debug);
24
25      X_opt = main_step(C, SIZ, is_debug);
26      fprintf("X_opt = \n");
27      disp(X_opt);
28
29      f_opt = get_f_opt(srcC, X_opt);
30      fprintf("f_opt = %d\n", f_opt)
31  end
32
33  function C = reduction_to_minimization(C, is_debug)
34      max_el = max(max(C));
35      C = max_el + -1 * C;
36
37      if is_debug
38          fprintf("Максимальный элемент: %d\n", max_el);
39          fprintf("Сведенная к задаче минимизации матрица
                стоимостей:\n");
40          disp(C);
41      end
42  end
43
44  function [C, SIZ] = prepare_step(C, is_debug)
45      if is_debug
46          fprintf("Подготовительный этап:\n\n");
47      end
48
49      C = sub_min_cols(C, is_debug);
50      C = sub_min_rows(C, is_debug);
51      SIZ = init_SIZ(C, is_debug);
52  end
53
54  function X_opt = main_step(C, SIZ, is_debug)
55      if is_debug
56          fprintf("\nОсновной этап:\n\n");
57      end
58
59      k = get_SIZ_k(SIZ);
60      [n, ~] = size (C);
```

Продолжение листинга 3.1

```
61
62      iters = n - k;
63      if is_debug && iters ~= 0
64          fprintf("k = %d < n = %d. Требуется %d итераций\n\n", k, n,
                iters)
65      elseif is_debug
66          fprintf("k = %d == n = %d. Итераций не требуется\n\n", k, n)
67      end
68
69      iter_num = 1;
70      while iters ~= 0
71          if is_debug
72              fprintf("Итерация %d\n\n", iter_num);
73          end
74
75          [C, SIZ] = improvement_SIZ(C, SIZ, is_debug);
76
77          iter_num = iter_num + 1;
78          iters = iters - 1;
79      end
80
81      X_opt = SIZ;
82  end
83
84  function mtx = sub_min_cols(mtx, is_debug)
85      min_els = min(mtx);
86      for i = 1 : length(min_els)
87          mtx(:, i) = mtx(:, i) - min_els(i);
88      end
89
90      if is_debug
91          fprintf("Минимальные элементы столбцов:\n");
92          disp(min_els);
93          fprintf("Матрица стоимостей после вычитания минимальных
                элементов из столбцов:\n");
94          disp(mtx);
95      end
96  end
97
```

Продолжение листинга 3.1

```
98 function mtx = sub_min_rows(mtx, is_debug)
99     min_els = min(mtx, [], 2);
100    for i = 1 : length(min_els)
101        mtx(i, :) = mtx(i, :) - min_els(i);
102    end
103
104    if is_debug
105        fprintf("Минимальные элементы строк:\n");
106        disp(min_els);
107        fprintf("Матрица стоимостей после вычитания минимальных
               элементов из строк:\n");
108        disp(mtx);
109    end
110 end
111
112 function SIZ = init_SIZ(C, is_debug)
113    [rows, cols] = size(C);
114    SIZ = zeros(rows, cols);
115
116    for i = 1 : rows
117        for j = 1 : cols
118            if C(i,j) == 0 && ~any(SIZ(:, j) == 1)
119                SIZ(i, j) = 1;
120                break
121            end
122        end
123    end
124
125    if is_debug
126        fprintf("Начальная СНН:\n");
127        print_SIZ(C, SIZ);
128        fprintf("Количество нулей СНН: k = %d\n", get_SIZ_k(SIZ));
129    end
130 end
131
132 function [C, SIZ] = improvement_SIZ(C, SIZ, is_debug)
133    [rows, cols] = size(SIZ);
134    marked_zeroes = zeros(rows, cols);
135    selected_cols = sum(SIZ);
```

Продолжение листинга 3.1

```
136     selected_rows = zeros(rows);
137     selected = update_selected(rows, cols, selected_rows,
            selected_cols);
138
139     if is_debug
140         fprintf("СНН с начально выделенными столбцами:\n");
141         print_marked_SIZ(C, SIZ, marked_zeroes, selected_rows,
                selected_cols)
142     end
143
144     marked_zero_pos = [-1 -1];
145     is_L_chain_builded = false;
146     while ~is_L_chain_builded
147         marked_zero_pos = get_zero_mark_pos(C, selected);
148
149         if marked_zero_pos(1) == -1
150             C = add_zeros_to_unselected(C, selected, selected_rows,
                    selected_cols);
151
152             if is_debug
153                 fprintf('\nСреди невыделенных элементов нет 0,
                        преобразуем матрицу:\n');
154                 print_marked_SIZ(C, SIZ, marked_zeroes,
                        selected_rows, selected_cols);
155             end
156
157             marked_zero_pos = get_zero_mark_pos(C, selected);
158         end
159
160         marked_zeroes = mark_zero(marked_zeroes, marked_zero_pos);
161         if is_debug
162             fprintf("\nОтметим 0' найденный невыделенный 0:\n");
163             print_marked_SIZ(C, SIZ, marked_zeroes, selected_rows,
                    selected_cols);
164         end
165
166         stared_zero_pos =
                get_stared_zero_pos_on_row_with_marked(SIZ,
                marked_zero_pos);
```

Продолжение листинга 3.1

```
167         if stared_zero_pos(1) == -1
168             is_L_chain_builded = true;
169         else
170             selected_rows(stared_zero_pos(1)) = 1;
171             selected_cols(stared_zero_pos(2)) = 0;
172             selected = update_selected(rows, cols, selected_rows,
                    selected_cols);
173 
174             if is_debug
175                 fprintf("\nПеренесем выделение со столбца на
                        строку, т.к. на строке с 0' есть 0*:\n");
176                 print_marked_SIZ(C, SIZ, marked_zeroes,
                        selected_rows, selected_cols);
177             end
178         end
179     end
180 
181     SIZ = build_L_chain(SIZ, marked_zeroes, marked_zero_pos,
            is_debug);
182 end
183 
184 function SIZ = build_L_chain(SIZ, marked_zeros, marked_zero_pos,
        is_debug)
185     if is_debug
186         fprintf("\nПостроена L-цепочка: ")
187     end
188 
189     i = marked_zero_pos(1);
190     j = marked_zero_pos(2);
191 
192     is_L_chain_ended = false;
193     while ~is_L_chain_ended
194         marked_zeros(i, j) = 0;
195 
196         stared_zero_row_pos = find(SIZ(:, j) == 1, 1);
197         SIZ(i, j) = 1;
198 
199         if is_debug
200             fprintf("0' (%d, %d) ", i, j);
```

Продолжение листинга 3.1

```
201         end
202
203         if isempty(stared_zero_row_pos)
204             is_L_chain_ended = true;
205         else
206             SIZ(stared_zero_row_pos, j) = 0;
207
208             if is_debug
209                 fprintf("-> 0* (%d, %d) -> ", stared_zero_row_pos,
                        j);
210             end
211
212             marked_zero_col_pos =
                    find(marked_zeros(stared_zero_row_pos, :) == 1, 1);
213             i = stared_zero_row_pos;
214             j = marked_zero_col_pos;
215         end
216     end
217
218     if is_debug
219         fprintf("\n\n");
220     end
221 end
222
223 function marked_zeroes = mark_zero(marked_zeroes, marked_zero_pos)
224     marked_zeroes(marked_zero_pos(1), marked_zero_pos(2)) = 1;
225 end
226
227 function C = add_zeros_to_unselected(C, selected, selected_rows,
      selected_cols)
228     h = find_h(C, selected);
229     [rows, cols] = size(C);
230
231     for i = 1 : cols
232         if selected_cols(i) == 0
233             C(:, i) = C(:, i) - h;
234         end
235     end
236     for i = 1 : rows
```

Продолжение листинга 3.1

```
237         if selected_rows(i) == 1
238             C(i, :) = C(i, :) + h;
239         end
240     end
241 end
242
243 function h = find_h(C, selected)
244     h = inf;
245     [rows, cols] = size(C);
246     for i = 1 : rows
247         for j = 1 : cols
248             if selected(j, i) == 0 && C(j, i) < h
249                 h = C(j, i);
250             end
251         end
252     end
253 end
254
255 function marked_zero_pos = get_zero_mark_pos(C, selected)
256     marked_zero_pos = [-1 -1];
257     [rows, cols] = size(C);
258     for j = 1 : cols
259         for i = 1 : rows
260             if selected(i, j) == 0 && C(i, j) == 0
261                 marked_zero_pos(1) = i;
262                 marked_zero_pos(2) = j;
263                 return
264             end
265         end
266     end
267 end
268
269 function stared_zero_pos =
    get_stared_zero_pos_on_row_with_marked(SIZ, marked_zero_pos)
270     i = marked_zero_pos(1);
271     [~, cols] = size(SIZ);
272
273     stared_zero_pos = [-1 -1];
274     for j = 1 : cols
```

Продолжение листинга 3.1

```
275         if SIZ(i, j) == 1
276             stared_zero_pos(1) = i;
277             stared_zero_pos(2) = j;
278             break
279         end
280     end
281 end
282
283 function selected = update_selected(rows, cols, selected_rows,
        selected_cols)
284     selected = zeros(rows, cols);
285     selected(:, selected_cols == 1) = 1;
286     selected(selected_rows == 1, :) = 1;
287 end
288
289 function k = get_SIZ_k(SIZ)
290     k = sum(sum(SIZ));
291 end
292
293 function print_SIZ(C, SIZ)
294     [rows, cols] = size(C);
295     for i = 1 : rows
296         for j = 1 : cols
297             if SIZ(i, j) == 1
298                 fprintf("\t%d*", C(i,j));
299             else
300                 fprintf("\t%d", C(i,j));
301             end
302         end
303         fprintf("\n");
304     end
305 end
306
307 function print_marked_SIZ(C, SIZ, marked_zeros, selected_rows,
        selected_cols)
308     [rows, cols] = size(C);
309     for i = 1 : rows
310         for j = 1 : cols
311             if SIZ(i, j) == 1
```

Продолжение листинга 3.1

```
312                 fprintf("\t%d*", C(i,j));
313             elseif marked_zeros(i, j) == 1
314                 fprintf("\t%d'", C(i,j));
315             else
316                 fprintf("\t%d", C(i,j));
317             end
318         end
319         if selected_rows(i) == 1
320             fprintf("\t+");
321         end
322         fprintf("\n");
323     end
324
325     for i = 1 : cols
326         if selected_cols(i) == 1
327             fprintf("\t+");
328         else
329             fprintf("\t ");
330         end
331     end
332     fprintf("\n");
333 end
334
335 function f_opt = get_f_opt(C, X_opt)
336     f_opt = sum(sum(C .* X_opt));
337 end
```

3.3 Результаты работы программы

```
Матрица стоимостей:
     7     4     3     8     2
     4     5     1     6     3
     8     4     5     7     2
     1     2     4     7     2
     3     9     9     2     5

Подготовительный этап:

Минимальные элементы столбцов:
     1     2     1     2     2
```

```
Матрица стоимостей после вычитания минимальных элементов из столбцов:
     6     2     2     6     0
     3     3     0     4     1
     7     2     4     5     0
     0     0     3     5     0
     2     7     8     0     3

Минимальные элементы строк:
     0
     0
     0
     0
     0

Матрица стоимостей после вычитания минимальных элементов из строк:
     6     2     2     6     0
     3     3     0     4     1
     7     2     4     5     0
     0     0     3     5     0
     2     7     8     0     3

Начальная СНН:
    6   2   2   6   0*
    3   3   0*  4   1
    7   2   4   5   0
    0*  0   3   5   0
    2   7   8   0*  3
Количество нулей СНН: k = 4

Основной этап:

k = 4 < n = 5. Требуется 1 итераций

Итерация №1

СНН с начально выделенными столбцами:
    6   2   2   6   0*
    3   3   0*  4   1
    7   2   4   5   0
    0*  0   3   5   0
```

```
    2   7   8   0*  3
    +       +   +   +

Отметим 0' найденный невыделенный 0:
    6   2   2   6   0*
    3   3   0*  4   1
    7   2   4   5   0
    0*  0'  3   5   0
    2   7   8   0*  3
    +       +   +   +

Перенесем выделение со столбца на строку, т.к. на строке с 0' есть 0*:
    6   2   2   6   0*
    3   3   0*  4   1
    7   2   4   5   0
    0*  0'  3   5   0   +
    2   7   8   0*  3
            +   +   +

Среди невыделенных элементов нет 0, преобразуем матрицу:
    4   0   2   6   0*
    1   1   0*  4   1
    5   0   4   5   0
    0*  0'  5   7   2   +
    0   5   8   0*  3
            +   +   +

Отметим 0' найденный невыделенный 0:
    4   0   2   6   0*
    1   1   0*  4   1
    5   0   4   5   0
    0*  0'  5   7   2   +
    0'  5   8   0*  3
            +   +   +

Перенесем выделение со столбца на строку, т.к. на строке с 0' есть 0*:
    4   0   2   6   0*
    1   1   0*  4   1
    5   0   4   5   0
    0*  0'  5   7   2   +
    0'  5   8   0*  3   +
```

```
            +       +

Отметим 0' найденный невыделенный 0:
    4   0'  2   6   0*
    1   1   0*  4   1
    5   0   4   5   0
    0*  0'  5   7   2   +
    0'  5   8   0*  3   +
            +       +

Перенесем выделение со столбца на строку, т.к. на строке с 0' есть 0*:
    4   0'  2   6   0*  +
    1   1   0*  4   1
    5   0   4   5   0
    0*  0'  5   7   2   +
    0'  5   8   0*  3   +
            +

Отметим 0' найденный невыделенный 0:
    4   0'  2   6   0*  +
    1   1   0*  4   1
    5   0'  4   5   0
    0*  0'  5   7   2   +
    0'  5   8   0*  3   +
            +

Построена L–цепочка: 0' (3, 2)

X_opt =
     0     0     0     0     1
     0     0     1     0     0
     0     1     0     0     0
     1     0     0     0     0
     0     0     0     1     0

f_opt = 10
```